

Lessons Learned – Darshan Ravindra Konnur

CS6650 Final Project

April 2026

What I worked on

My primary areas were the order-worker service (SQS consumer, idempotent DynamoDB persistence, AWS smoke test), the quantity-support fix in `flash-sale-api`, and the later Experiment 2 additions around inventory strategy modes and the Locust-based load harness.

What went well

Decoupling acceptance from persistence was the right architectural choice. The order-worker let the request path stay simple: once the API reserved inventory and published a message, persistence could happen asynchronously without forcing Redis and DynamoDB into the same transaction. That is exactly the kind of queue-based separation we talked about throughout the course, and in the final system it paid off: Experiment 3 showed peak SQS depth staying at 0 once admission control was wired correctly.

The AWS smoke test made the worker testable as a system, not just as a package. It is easy to unit test order parsing and DynamoDB writes in isolation; it is harder to prove that `purchase → SQS → worker → DynamoDB` actually works against the deployed environment. The smoke test closed that gap and made it possible to validate the worker without guessing from logs.

Adding explicit inventory strategy modes made Experiment 2 cleaner. Once the strategy selector existed, we could compare atomic decrement, optimistic locking, and Lua without maintaining separate branches or three slightly different handlers. That made the experiment easier to reason about and less error-prone.

What went wrong

I underestimated how much “correct but lossy” behavior matters. The optimistic-locking path never over-sold, but the experiment showed that under contention it could leave most of the inventory unsold unless you retry aggressively. That was a useful reminder that correctness alone is not enough in distributed systems; if the coordination mechanism causes the business outcome to collapse, it is still the wrong design.

The project depended too much on late-stage integration. The order-worker, `flash-sale-api`, and waiting-room each made sense on their own, but several of the most important bugs were only visible when the full chain ran together: the missing admission-token enforcement, the waiting-room race, and the local endpoint assumptions in the parity stack. We should have been doing small end-to-end checks much earlier.

The original plan assumed the tooling would already be in place. Locust and the AWS/local harnesses were straightforward once they were there, but the time to get the environment aligned came late and compressed the experiment window. The lesson is that load-generation infrastructure is part of the experiment, not a prerequisite that can be hand-waved away.

How the course concepts applied

- At-least-once delivery and idempotency. The order-worker is built around the idea that SQS may redeliver messages, so DynamoDB writes need to be idempotent. This is one of the most practical distributed systems ideas from the course: if retries are possible, duplicate processing must be safe.
- Asynchronous decoupling. The worker exists because the user-facing request path should not block on database persistence. Using SQS as the hand-off boundary keeps the critical path short and moves slower work to a component that can scale independently.
- Tradeoffs between correctness mechanisms. Experiment 2 made the abstract lecture point concrete: atomic server-side operations and Lua scripts behaved well under contention, while optimistic coordination preserved safety but degraded availability badly.

What I would do differently

1. Add an end-to-end smoke path during the first week, not after most of the service code is already written.
2. Treat load-test tooling as a first-class deliverable and verify it early, the same way we verify the application itself.
3. Add more explicit failure-mode tests for duplicate messages, publish failures, and retry behavior instead of assuming the happy path is representative.